directive(自定义指令)

除了核心功能默认内置的指令（v-model 和 v-show），Vue 也允许注册自定义指令。注意，在 Vue2.0 中，代码复用和抽象的主要形式是组件。然而，有的情况下，你仍然需要对普通 DOM 元素进行底层操作，这时候就会用到自定义指令。

全局自定义指令：Vue.directive

局部自定义指令：directives

```
//获取焦点
<div id="app">
账号：<input type="text"  />
密码：<input v-focs.aa type="text" />
</div>
传入的两个参数：id就是指的id( v-if 、v-for),definition是指的定义对象，
其中定义对象可以提供一些钩子函数
Vue.directive('id',{})
// 定义全局指令
Vue.directive('focs',{
        /*
            el：指令属性所在的标签元素
            binding:包含指令相关数据的对象
         */
        inserted:function(el,binding,vnode){
            console.log(el)
            console.log(binding)
            console.log(vnode)
            el.focus()
        }
    })
    new Vue({
        el:'#app',
        data:{
            msg:"刷新页面获取焦点"
        },
    })

//大小写转换
    <div id="app">
        <h5 v-upper*text="u"></h5>
        <h5 v-lower-text="l"></h5>
    </div>

    <div id="a" style="border: 1px solid #FF0000;">
        <h5 v-upper*text="u"></h5>
        <h5 v-lower-text="l"></h5>
    </div>
```

```
//直接写function代表了bind和updata
Vue.directive('upper*text',function(el,binding){//转换成大写
    console.log(el,binding)

    el.innerHTML=binding.value.toUpperCase()
  })
  new Vue({
    el:'#app',
    data:{
      msg:"刷新页面获取焦点",
      u:"i like you",
      l:"ABC DDD"
    },
    directives:{//定义局部（私有的）
      'lower-text' :function(el,binding){//转换成小写
        el.innerHTML=binding.value.toLowerCase()
      }
    }
  })

  new Vue({//不可以调用私有的局部指令，会报错。
    el:"#a",
    data:{
      msg:"刷新页面获取焦点",
      u:"i like you",
      l:"ABC DDD"
    },
  })

//js表达式
  <div id="app">//定义一个指令，动态传递参数（文本，颜色）
    <h3 v-my-text="{name:'周生生',color:'blue'}"></h3>
    <h3 v-my-text="{name:'Terrrance',color:'orange'}"></h3>
  </div>

  new Vue({
    el:"#app",
    data:{

    },
    // 注册局部自定义指令
    directives:{
      'my-text':function(el,binding){
        console.log(el,binding)
```

```
            // 设置文本及背景颜色
            el.innerHTML=binding.value.name
            el.style.backgroundColor=binding.value.color
        }
    }
})
```

钩子函数

bind 只调用一次，指令第一次绑定到元素时调用。在这里可以进行一次性的初始化设置。

inserted 被绑定元素插入父节点时调用（仅保证父节点存在，但不一定已被插入文档中）。

update 所在组件的 VNode 更新时调用，但是可能发生在其子 VNode 更新之前。指令的值可能发生了改变，也可能没有。但是你可以通过比较更新前后的值来忽略不必要的模板更新。

componentUpdated 指令所在组件的 VNode 及其子 VNode 全部更新后调用。

unbind 只调用一次，指令与元素解绑时调用。

钩子函数的参数

el 指令所绑定的元素，可以用来直接操作 DOM。

binding 一个对象，包含一下这些属性(name、value、oldValue、expression、arg、modifiers后缀标识符)

vnode vue 编译生成的虚拟节点。

oldVnode 上一个虚拟节点，仅在 update 和 componentUpdated 钩子中可用。

过滤器

Vue.js允许你自定义过滤器,可被用于一些常见的文本格式化,过滤器可以用在两个地方 : 双花括号插值和v-bind表达式（后者从 2.1.0+ 开始支持）. 过滤器应该被添加在JavaScript表达式的尾部,由管道符 : | 指示,表达式的值将作为形参传入到filter中。

全局过滤器：Vue.filter

局部自定义指令：filters

```
<div id="app">
<p>苹果价格 :{{price1 | formatPrice }} 元/斤</p>
<!-- 传递参数 过滤器是 JavaScript 函数，因此可以接收参数-->
<p>梨子价格 :{{price2 | formats("元/斤") }} </p>
</div>
// 定以全局过滤器
//当全局过滤器和局部过滤器重名时，会采用局部过滤器。
Vue.filter('formatPrice',function(p){
    console.log(p)
    return (p/100).toFixed(1);
})
new Vue({
    el:"#app",
```

```
        data:{
            price1:1888, price2:1668,price3:1778,price4:1998,
        },
        filters:{ // 私有的过滤器(局部过滤器)
            /*
                过滤器使用括号执行的时候,会传入相应的参数
                ，输入的参数会作为第二个参数传递给过滤器
             */
            formats(p,m){ //m（形参）函数传递过来的参数
                console.log(p,m)
                return "$"+(p/100).toFixed(2) +m
            }
        }
    })
```

过滤器传参：

```
<p>梨子价格 :{{price2 | formats(″元/斤″) }} </p>
```

过滤器会把 | 前面的内容作为第一个参数，如果过滤器只需要处理这一个参数是可以不加括号，如果过滤器需要传递其他参数来动态

改变逻辑的话，可以加括号执行，并传入相应的参数，输入的参数会作为第二个参数传给过滤器。

//过滤器管道式链接

```
    <div id="app">
    <input type="text" name="msg" v-model="msg"/>
    <div style="margin-top:20px;">
        过滤数字: <p>{{msg | fNum |arr}}</p>
    </div>
    password<input type="text"  :value="msg | fNum" />

    new Vue({
        el:"#app",
        data:{
            msg:''
        },
        filters:{
            fNum(value){//过滤数字
                console.log(value)
                return value.replace(/[0-9 a-z A-Z]/g,'*');
            },
            arr(val){
                // 将字符分割成数组
                return val.split('');
            }
        }
    })
```